🐾 Pet-Care Connecting System

Viva Preparation — Part 1: Project Overview & Architecture

Q1 What is your project about?

Our project is a **Pet-Care Connecting System —** a web application that helps pet owners manage their pets' health online. They can register pets, book vet appointments, track medications, get AI-powered health checkups, chat with an AI assistant, browse adoption listings, learn from educational content, and find nearby vets. The whole idea is one platform for everything pet-related.

Q2 What technologies did you use?

Layer	Technology
Backend	Java 17, Spring Boot 3.4.3, Spring Security, Spring Data JPA, Hibernate
Database	MySQL 8.0 (Docker container)
Frontend	React 18, TypeScript, React Router, Axios
Auth	JWT (JSON Web Token) with BCrypt password hashing
AI / ML	Google Gemini 2.0 Flash API (image + text), Groq LLaMA 3.3 70B (chatbot + fallback)
Tools	Lombok, Docker Compose, Maven

Q3 How is the project structured?

Two separate folders:

Folder	Purpose
`pet-care-backend/`	Spring Boot REST API server
`pet-care-frontend/`	React TypeScript user interface

Backend packages:

```
com.petcare.backend
├── config/         → Security config, CORS, data seeder
├── controller/     → 11 REST controllers (API endpoints)
├── dto/            → Data Transfer Objects
├── entity/         → 9 JPA entities (database tables)
├── repository/     → 9 JPA repository interfaces
├── security/       → JWT filter + JWT utility service
├── service/        → 12 service interfaces
└── service/impl/   → 10 service implementations
```

Frontend structure:

```
src/
├── api/            → Axios config (base URL + JWT interceptor)
├── components/     → Navbar, Sidebar, AppLayout, ProtectedRoute
├── context/        → AuthContext (login state)
├── pages/          → 15 page components
└── App.tsx         → Main routing file
```

Q4 What are the main features?

1. **User Authentication** — Register, login, JWT sessions
2. **Pet Management —** Add/edit/delete pets with details
3. **Vet Appointments** — Book vet visits, video calls, pet sitting, emergency
4. **AI Health Consultation —** Upload photo + symptoms → get AI diagnosis → vet reviews
5. **AI Chatbot** — Ask pet health questions, instant answers
6. **Medication Tracker —** Track medications (name, dosage, frequency, dates)
7. **Surgical Procedures** — Schedule surgeries with pre-op/post-op notes
8. **Pet Adoption —** List and browse pets for adoption
9. **Educational Content** — Articles/videos on pet care topics
10. **Nearby Vets —** Find vet clinics near your location
11. **Notifications** — Vets/admins send alerts to users
12. **Admin Panel —** Manage users, roles, all records

Q5 What user roles exist?

Role	Permissions
`USER`	Register pets, book appointments, get AI diagnosis, chat, adopt, medications
`VET`	All user features + review AI consultations, add final diagnosis, send notifications

`ADMIN`	Full access — manage users, delete records, view everything, send notifications

Roles are stored as an enum in the `User` entity and enforced with Spring Security on backend + `RoleProtectedRoute` on frontend.

Q6 Explain the database design.

We have **9 tables** managed by JPA/Hibernate:

Table	Key Fields	Relationships
`users`	id, firstName, lastName, email, password, role	—
`pets`	name, species, breed, age, weight, medicalHistory	owner → User
`appointments`	appointmentTime, reason, status, meetingLink, serviceType	pet → Pet, owner → User, vet → User
`consultations`	userDescription, aiPreliminaryDiagnosis, vetFinalDiagnosis, status	pet → Pet, owner → User, vet → User
`medications`	name, dosage, frequency, startDate, endDate, notes	pet → Pet, owner → User
`surgical_procedures`	procedureName, preOpInstructions, surgeryDateTime, postOpNotes, status	pet → Pet, owner → User, vet → User
`adoption_listings`	petName, species, breed, gender, description, imageUrl, location, status	listedBy → User, adoptedBy → User
`educational_content`	title, content, category, type, mediaUrl	—
`notifications`	title, message, type, isRead, createdAt	recipient → User, sender → User

We used `@ManyToOne` JPA relationships. Hibernate auto-creates tables because `ddl-auto=update`.

Q7 How does authentication work?

1. User registers → password hashed with **BCrypt** → stored in DB
2. User logs in → backend validates credentials → generates **JWT token** → sends back
3. Frontend stores JWT in `localStorage`
4. Every API request includes JWT in the `Authorization: Bearer <token>` header
5. `JwtAuthFilter` intercepts every request → validates token → sets SecurityContext
6. If token is missing/expired → request rejected with 401

Sessions are **stateless** (`SessionCreationPolicy.STATELESS`) — no server-side session, JWT carries all info. Token expires after **24 hours**.

Q8 How does frontend routing work?

We use **React Router v6**:

- **Public routes** (no login): `/` , `/login` , `/register` → shown with top `Navbar`
- **Protected routes** (login required): Dashboard, Pets, Medications, Chat, etc. → wrapped in `ProtectedRoute` + uses sidebar `AppLayout`
- **Role-protected routes**: Admin page (ADMIN only), Notifications (VET/ADMIN only) → wrapped in `RoleProtectedRoute`

Q9 What is CORS and how did you handle it?

CORS = Cross-Origin Resource Sharing. Since frontend (`localhost:3000`) and backend (`localhost:8080`) run on different ports, the browser blocks cross-origin API calls by default.

We configured it in `SecurityConfig.java` : allow origin `http://localhost:3000` , methods GET/POST/PUT/DELETE, all headers, and credentials enabled.

Q10 What is Lombok and why did you use it?

Lombok generates boilerplate code at compile time:

- `@Data` → getters, setters, toString, equals, hashCode
- `@Builder` → builder pattern for creating objects
- `@NoArgsConstructor` / `@AllArgsConstructor` → constructors
- `@RequiredArgsConstructor` → constructor for `final` fields (used for dependency injection)

Without Lombok each entity class would be 3x longer.

Q11 How does the frontend talk to the backend?

Using **Axios** (configured in `axiosConfig.ts`):

- Base URL set to `http://localhost:8080`
- Request interceptor automatically attaches JWT token to every request
- Pages call `api.get(...)` or `api.post(...)` to hit backend REST endpoints

- JSON responses are directly rendered in React components

Q12 What design patterns did you use?

Pattern	Where
MVC	Controllers → Services → Repositories
Repository	Spring Data JPA abstracts SQL
Builder	Lombok `@Builder` for entities
DTO	Data Transfer Objects between layers
Service Layer	Business logic in service impls, thin controllers
Filter Chain	JWT filter in Spring Security chain
Context	React Context API for auth state

Q13 What is Docker Compose used for?

Only for the **MySQL database**. The `docker-compose.yml` defines a MySQL 8.0 container with root password, database name `petcare_db`, port 3306, persistent volume, and health check. Run `docker-compose up -d` and the DB is ready — no manual MySQL install needed.

Q14 What appointment service types exist?

Type	Description
`VET_VISIT`	In-person vet checkup
`VIDEO_CONSULTATION`	Online video call with meeting link
`PET_SITTING`	Pet care while owner is away
`EMERGENCY`	Urgent care booking

Status: `SCHEDULED → COMPLETED / CANCELLED`

Q15 How does the notification system work?

- **Targeted —** send to a specific user (e.g., appointment reminder)
- **Broadcast** — send to all users (if `recipientUser` is null)
- Types: `INFO`, `WARNING`, `REMINDER`, `ALERT`
- Tracks read/unread status with badge count in sidebar
- Only VET and ADMIN roles can send notifications

Q16 What challenges did you face?

- **CORS issues —** fixed with proper `CorsConfigurationSource` in Spring Security
- **JWT + Spring Security 6** — had to use new lambda-style config (old `WebSecurityConfigurerAdapter` removed)
- **AI API rate limits —** added Groq as fallback when Gemini fails
- **Base64 image handling —** careful mime type handling for Gemini API

Q17 What would you improve with more time?

- Real-time notifications with WebSocket
- Payment integration for bookings
- Mobile app with React Native
- Vaccination schedule tracking
- Cloud file storage (AWS S3) instead of base64
- Unit tests and integration tests
- Cloud deployment with CI/CD pipeline

Pet-Care Connecting System — Viva Preparation Part 1 — Page End